Partial Assembly by Projection: an inexact matrix-free apply for the SFEM code generator

Abstract

The matrix-free apply of a nonlinear elasticity operator carries a quadrature loop because the material tangent varies across each element. Projecting that tangent onto the constants separates the loop into a small per-element object and a tensor that belongs to the reference element alone. The reference tensor can then be integrated symbolically once, at generation time, and it turns out to be far smaller than its size suggests: it is a Gram matrix, so its rank is the dimension of the span of the element's gradient functions. For a linear simplex that rank is one, and the construction reduces exactly to the loperand. This note states the technique, the compression, what the code generator emits, how it vectorises, and the measurements taken so far.

1 The exact apply, and why it has a loop

Let K be an element with reference cell $\widehat{K}$ and affine or isoparametric map $\boldsymbol{x}(\boldsymbol{\xi})$, Jacobian J, and $\mathrm{d}V = \det J \,\mathrm{d}\boldsymbol{\xi}$. For a hyperelastic energy with first Piola stress $P(F)$, the matrix-free action of the second form on an increment h is

$$(\mathcal{H}h)_{i,p} = \int_K S_{ijkl}\big(F(\boldsymbol{x})\big)\,\frac{\partial h_k}{\partial x_l}\,\frac{\partial \varphi_p}{\partial x_j}\,\mathrm{d}V, \qquad S_{ijkl} = \frac{\partial P_{ij}}{\partial F_{kl}}, \tag{1}$$

with i the output component, p the test function, and φ_p the basis.

Pull the derivatives back to the reference cell with $\partial/\partial x_j = \sum_n J^{-1}_{nj}\,\partial/\partial \xi_n$ and collect the geometry into the tangent:

$$\overline{S}_{ikmn} = \sum_{j,l} S_{ijkl}\, J^{-1}_{nj}\, J^{-1}_{ml}\, \det J, \tag{2}$$

so that (1) becomes

$$(\mathcal{H}h)_{i,p} = \int_{\widehat{K}} \overline{S}_{ikmn}(\boldsymbol{\xi})\,\frac{\partial h_k}{\partial \xi_m}\,\frac{\partial \varphi_p}{\partial \xi_n}\,\mathrm{d}\boldsymbol{\xi}. \tag{3}$$

This $\overline{S}$ is the *loperand*: the flux differentiated once and already contracted with the geometry, so that the element vector is nothing but differences of its contractions. It is the same object SFEM's hand-written kernels build, and the same one `plans/loperand.py` isolates from a material's flux.

Everything under the integral except $\overline{S}$ is a property of the reference element. What forces a quadrature loop is that $\overline{S}$ depends on $\boldsymbol{\xi}$, because F does.

2 The projection

Replace $\overline{S}$ by its L^2 projection onto the constants on K, that is by its element average $\overline{S}^0 = |\widehat{K}|^{-1}\int_{\widehat{K}} \overline{S}$. It leaves the integral and the sum separates:

$$(\mathcal{H}h)_{i,p} \;\approx\; \overline{S}^0_{ikmn} \sum_j h_{k,j}\,\overline{W}_{jmpn}, \qquad \overline{W}_{jmpn} = \int_{\widehat{K}} \frac{\partial \varphi_j}{\partial \xi_m}\,\frac{\partial \varphi_p}{\partial \xi_n}\,\mathrm{d}\boldsymbol{\xi}. \tag{4}$$

$\overline{S}^0$ is one small object per element, computed once from the state. $\overline{W}$ belongs to the element and is integrated exactly, symbolically, at generation time. *The quadrature loop leaves the kernel entirely.*

Where the approximation is, and where there is none

On an affine simplex the deformation gradient is constant over the cell, so $\overline{S}$ is constant, so projecting it onto the constants returns it unchanged and (4) is an identity rather than an approximation. This holds for a nonlinear material as much as a linear one: it is a statement about the geometry, not about P.

That is the load-bearing fact of the whole construction, because it gives an exact reference to test against. On TRI3 and TET4 the projected kernel must reproduce the exact one; only on curved or higher-order elements is there an error, and there it must be measured rather than assumed.

Symmetry

The element matrix entry implied by (4) is $\overline{S}^0_{ikmn}\overline{W}_{jmpn}$. Requiring it to be symmetric under exchanging the two (component, node) pairs, and using $\overline{W}_{jmpn} = \overline{W}_{pnjm}$, forces

$$\overline{S}^0_{ikmn} = \overline{S}^0_{kinm}. \tag{5}$$

The pair that exchanges is (i, n) against (k, m): the output component with the *test* direction, and the increment component with the *increment* direction. Packing on (i, k) against (m, n) instead is the plausible-looking error, and it is invisible to any test that compares the construction against itself. Under (5), $\overline{S}^0$ has 45 independent components in three dimensions and 10 in two.

The derivation above assumed the element matrix is symmetric, and that is a property of the *formulation*, not of the method. A material written as an energy has a flux that is a gradient, so its tangent is a Hessian and (5) holds by equality of mixed partials. A material written as a residual has no such guarantee: its flux is not a gradient, its tangent is a Jacobian, and it need not be symmetric at all. Kelvin–Voigt viscosity is not — measured, $\max_{ijkl} |A_{ijkl} - A_{klij}| = 0.25$ — so folding it into 45 components averages its antisymmetric part away and yields a *different operator*: 5.3×10^{-2} relative error on TET4, an element where the projection is provably exact.

The generator therefore decides symmetry per material, symbolically, and stores 45 or 81 accordingly. It defaults to unsymmetric, because the two errors are not comparable: storing a full square for a symmetric tangent wastes memory, while storing half of an unsymmetric one is silently wrong.

3 Compression of the reference tensor

3.1 What it contains

$\overline{W}$ is integrated exactly, so its entries are rationals. There are very few of them, and on the simplices most are zero.

3.2 Rank, which is where the structure really is

The zeros are a symptom. $\overline{W}$, read as a matrix on the pair index (j, m), is the Gram matrix of the element's gradient functions $\{\partial\varphi_j/\partial\xi_m\}$ in $L^2(\widehat{K})$. Its rank is therefore the dimension of the span of those functions and nothing more.

A linear simplex is rank one because its gradients are constants. Rank one is precisely the loperand of Section 1: one object per element, contracted once. So the rank factorisation is not

element	entries	distinct values	zeros	projection
TRI3	36	3	20	exact
TET4	144	3	108	exact
QUAD4	64	6	0	inexact
HEX8	576	10	0	inexact
TET10	900	9	519	inexact

Table 1: The symbolically integrated reference tensor. At most ten distinct rationals for any element.

element	order	rank	span of the gradient functions
TRI3	6	1	constants
TET4	12	1	constants
QUAD4	8	3	$1,\ \xi_0,\ \xi_1$
HEX8	24	7	1 and the six bilinear monomials
TET10	30	4	$1,\ \xi_0,\ \xi_1,\ \xi_2$

Table 2: Rank of the reference tensor, which equals the dimension of the span of the element's gradient functions exactly.

a second idea beside the loperand; it is the same structure carried to elements whose gradients are not constant.

Factor exactly over the rationals,

$$\overline{W} = C^{\mathsf{T}} X C, \qquad C \in \mathbb{Q}^{r \times Nd}, \qquad X \in \mathbb{Q}^{r \times r}, \tag{6}$$

with r the rank, N the node count and d the dimension. The factorisation is checked against the tensor it factors when it is built.

4 The generated kernel

4.1 Staging

Substituting the factorisation into (4) and naming each intermediate gives four stages:

$$P_{kam} = \sum_{j} C_{a,(j,m)}\, h_{k,j} \qquad \text{the increment, compressed} \tag{7}$$

$$Y_{ian} = \sum_{k,m} \overline{S}^{0}_{ikmn}\, P_{kam} \qquad \text{the tangent applied} \tag{8}$$

$$Q_{ibn} = \sum_{a} X_{ab}\, Y_{ian} \qquad \text{the middle factor} \tag{9}$$

$$\mathrm{out}_{ip} = \sum_{b,n} C_{b,(p,n)}\, Q_{ibn} \qquad \text{expanded back to nodes} \tag{10}$$

Every sum skips its zero coefficients, so a zero of C or X never enters an expression rather than being cancelled out of one later.

The staging is the point and not an implementation detail. Expanding the four stages into one expression per output and letting common-subexpression elimination rediscover the structure returns the dense operation count exactly; the saving lives in keeping the intermediates named.

4.2 Choosing the contraction

Section 5 stages the action through the rank factorisation. That is not always the cheaper arrangement. The factorisation has to compress the increment and expand the result, and where $\overline{W}$'s rank is a large fraction of its order, those stages cost more than they save. The alternative contracts $\overline{W}$ against the increment directly,

$$G_{kmpn} = \sum_j \overline{W}_{jmpn}\, h_{k,j} \tag{11}$$

$$\mathrm{out}_{ip} = \sum_{k,m,n} \overline{S}^0_{ikmn}\, G_{kmpn}, \tag{12}$$

in which $\overline{S}^0$ appears once, in a single 27-term contraction per output, and there is nothing to compress or expand.

Which wins is a property of the element, so the generator measures both and emits the cheaper. Counted after common-subexpression elimination:

element	staged	gradient-first	rank	chosen
TET4	207	342	1 of 12	staged
TET10	1380	1605	4 of 30	staged
HEX8	2610	1731	7 of 24	gradient-first

Table 3: Operations for the two contraction orderings. HEX8's rank is high enough that the factorisation stops paying.

On HEX8 this takes the emitted apply from 2732 operations to 1801, against a hand-written `hex8_SdotHdotG` at 1558, and lifts its measured rate from about 100 to about 132 MDOF/s.

The counting has to be done *after* elimination. Measured on the unsimplified expressions, gradient-first looks worse on every element, which is how it was first dismissed.

4.3 Fold the constants, do not tabulate them

The entries of $\overline{W}$ are known at generation time, so they are emitted as literals. The tempting alternative — store the distinct values in a runtime table and index it — compresses the memory and destroys the arithmetic: the constants stop being constants, the compiler can no longer fold them, and every multiply by a zero entry survives into the emitted code.

This is not hypothetical. The existing prototype (`python/codegen/neohookean_partial_assembly.py`) emits, for TET4, a table holding exactly 1, -1 and 0, and its kernels then multiply by the zero entry twelve times each; the reported "97.9% memory saving" buys 141 stored numbers that were 0 and ± 1 at a cost of several hundred floating-point operations.

element	rank/order	dense, folded	staged, folded	runtime table
TRI3	1/6	96	**52**	244
TET4	1/12	486	**207**	1467
QUAD4	3/8	238	238	431
HEX8	7/24	4059	**2610**	6330
TET10	4/30	4018	**1380**	8349

Table 4: Operations in the element apply, counted after common-subexpression elimination. Preliminary: these are symbolic operation counts, not measured run times.

Table 4 says where the staging pays. The saving tracks the rank against the order: TET10 at 4/30 gains 2.9× and TET4 at 1/12 gains 2.4×, while QUAD4 at 3/8 comes out exactly even

— not enough structure to beat common-subexpression elimination on the dense form. Against the runtime-table form the improvement is 7.1× on TET4 and 6.1× on TET10.

5 Vectorisation

The generated kernels process a block of `VECTOR_SIZE` elements at a time, with the lane loop innermost and marked `#pragma omp simd`. The staged form suits that layout well, and for reasons worth stating separately.

The reference tensor is lane-invariant. C and X are compile-time rationals, identical for every element and therefore for every lane. Folded, they appear as immediates: no load, no broadcast, and a multiply by ± 1 or by a power of two is folded away entirely. Behind a runtime table the same values become a memory reference whose address does not depend on the lane — a broadcast load rather than an immediate — and, worse, an opaque value that no longer permits folding. This is the arithmetic half of the cost in Table 4.

The tangent is a per-element stream. $\overline{S}^0$ is 45 numbers per element in three dimensions, stored one component-array per index in the structure-of-arrays layout the framework already uses for geometry. Each component is then a unit-stride read across the lane loop, which vectorises without gather.

The intermediates are lane-width vectors. P, Y and Q are per-element scalars, so in a blocked kernel each becomes one vector register. Their counts are the stage sizes in Table 5, and they are the binding constraint: on HEX8, $63 + 63 + 63$ live intermediates at a lane width of eight doubles will not stay in registers, and the kernel will spill. Rank is what controls this, since all three stage sizes are $d^2 r$.

The output scatter is unchanged. The last stage writes one value per node per component, exactly as the existing apply kernels do, so it reuses the same scatter — atomic on the plain traversal, and plain accumulation into thread-private scratch on the packed one.

element	rank	$\lvert P\rvert$	$\lvert Y\rvert$	$\lvert Q\rvert$	outputs
TRI3	1	4	4	4	6
TET4	1	9	9	9	12
QUAD4	3	12	12	12	8
HEX8	7	63	63	63	24
TET10	4	36	36	36	30

Table 5: Live intermediates per element in each stage. In a blocked kernel each is one vector register, so these are the register-pressure figures.

A consequence worth drawing out: on the elements where the projection is exact the register pressure is also lowest, and on HEX8 — where the projection is an approximation — it is highest. The two difficulties arrive together, and they are the same difficulty: rank controls both. This is why HEX8 is also the element on which the staged form loses to the direct contraction, which needs no intermediates at all. A rank-truncated factorisation (keeping $r' < r$ modes) remains the knob for elements where staging still wins, at the price of a second and larger approximation.

6 Verification

Each stage can be wrong in a way the next cannot see, so the construction is checked at four independent points.

1. **The symbolic integration**, against four-point Gauss–Legendre quadrature on QUAD4 and HEX8, entry by entry: agreement to 10^{-16}.
2. **The factorisation**, against the tensor it factors: $C^{\mathsf{T}} X C = \overline{W}$ exactly, over the rationals.
3. **The action**, against (1) integrated directly on a concrete tetrahedron, for linear elasticity and for neohookean Ogden.
4. **The generated operator**, inside SFEM, driven as a solver drives it: `initialize`, `apply`, `inexact_update`, `inexact_apply`.

Only the third can catch a wrong index convention, and it did: the symmetry (5) had been implemented as (i, k) against (m, n), which the first two checks and any comparison of the staged form against the dense form all pass, because they share the error.

Only the fourth can catch anything about integration. It found that the emitted operator source omitted the C ABI header, leaving every kernel signature unparseable; that the affine geometry cache the split reads was not built unless some *other* path had asked for it; and that `inexact_supported()` reported support on meshes where the geometry cache cannot exist. None of these is visible in the emitted text.

How large is the projection error where it is not zero

Section 3 leaves the curved case open: on TET10 and HEX8 the projection is an approximation and its error has to be measured. It is measured by warping the displacement — an increasing perturbation of the reference configuration, checked at each step to keep $\det F > 0$ — and comparing the projected neohookean apply against the exact one at each severity.

warp w	0	0.4	1.0	1.6
TET4	1e-15	1e-15	1e-15	1e-15
TET10	2.4e-14	1.0e-02	—	4.3e-02
HEX8	2.7e-15	—	—	3.7e-04

Table 6: Relative error of the projected apply against the exact one, $n = 16$. The $w = 0$ column is the control: it recovers the exactness proof of Section 3 and shows the sweep is measuring curvature and not the harness. TET4 is flat across the sweep, as it must be.

TET10 is two orders worse than HEX8 at comparable warp. Both are quadratic in the geometry, but TET10 carries a quadratic *displacement* whose gradient varies linearly over the cell, and it is that variation the projection onto the constants discards. The error is a real property of the approximation, not a defect: it falls under refinement at $O(h^{1.07})$ (see Methodology for why the first measurement of this appeared not to converge), so the projected operator converges to the exact one at the rate the discretisation itself does.

7 Methodology

Several of the numbers in earlier drafts of this note were wrong. The errors were not in the kernels, and they are worth recording, because each is a way of measuring the wrong thing while appearing to measure the right one.

Time the kernel, not the harness. The first throughput figures put a serial `std::fill` of the output arrays inside the timed region, and wrapped a single call per sample so that each paid OpenMP team startup and a cold cache. Neither is part of the operator. The distortion

is invisible at one thread and grows with the thread count — the worst possible shape for a scaling study: the single-thread figures barely moved when it was fixed (2.52 to 2.68 MDOF/s) while the eight-thread figures moved by half (13.04 to 18.75). Reported scaling was 5.4× where it is really 7.0×.

A ratio can be flat while the thing it measures converges. Measured against a *smooth* increment, the TET10 projection error sat at 1.0×10^{-2} across an eightfold refinement and looked like a defect. It is not. For a smooth field the assembled $(Hh)_p$ is a discrete second derivative, so neighbouring elements cancel and the denominator collapses by an extra order, while the per-element projection errors carry independent signs and do not cancel at all. Against a white-noise increment the same kernel converges at $O(h^{1.07})$. Before concluding that an approximation fails to converge, re-measure it with a random vector.

Do not measure at a convenient mesh size. The stored tangent is `metric_tensor_t`, which is `float`. On a mesh whose spacing is a power of two the tangent entries are exactly representable and the difference from the exact apply collapses to 10^{-14}, which looks like proof of something far stronger than is true. The same kernels at resolution 12:

element	$n = 8$	$n = 12$	$n = 16$
TET4	6.1e-16	1.2e-07	1.8e-15
TET10	1.4e-14	1.0e-05	4.8e-14
HEX8	–	6.8e-07	–

So the split reproduces the exact apply to the precision of its store, and that precision is element-dependent: TET10's quadratic basis gives its tangent a wider dynamic range and costs about two more digits than TET4 for the same `float` store. That is an input to choosing a store precision, not a tolerance to widen.

A conclusion from one machine is a conclusion about that machine. On an Apple laptop every column fell by a fifth past eight threads, and the atomic scatter looked like the scalability limit. It is not: the laptop has eight performance cores and two efficiency cores, `schedule(static)` hands them equal chunks, and the loop waits for the slow ones. On 72 homogeneous Grace cores the same kernels scale essentially linearly.

8 Measured performance

Two frames, and they are not interchangeable. The first calls the generated kernels directly with the geometry precomputed; the second goes through `Op::apply`, which is the path a solve actually takes. At 170,373 degrees of freedom the same kernel reads about 5 MDOF/s in the first and 1.46 in the second. Only the second is comparable with the library.

Against the library's own operators

Measured with SFEM's `bench_op`, a spike copy of which reports setup and application separately. `build64`, eight threads, ∼10.4M dof.

The assemblies, measured separately: TET10 0.392 s for the generated split against 0.653 s hand-written; HEX8 level at 0.414 against 0.427. So on TET10 the generated split beats hand-written partial assembly on both halves. On HEX8 it doubles the exact generated kernel but remains behind the hand-written one, and since the assemblies are level the whole gap is the contraction.

element	hand-written PA	exact generated	generated split	vs. hand-written
TET10	137.4	27.2	219.6	1.60× faster
HEX8	162.9	47.5	100.2	1.63× behind
TET4	(none)	47.9	116.9	—

Table 7: MDOF/s for the apply. "Hand-written PA" is `NeoHookeanOgden`, which enables partial assembly by hand for HEX8 and TET10 and not for TET4 — which is exactly where the generated kernels were ahead and behind before this work.

Break-even needs a reference that performs no setup of its own. The hand-written operator assembles too, so the exact generated apply is the honest denominator: against it, TET10 breaks even at 1.03 applies per tangent.

Threads

	1	8	32	64	72
Apple, exact	2.68	18.75	—	—	—
Apple, stored f16	7.61	55.15	—	—	—
Grace, exact	1.33	10.68	41.65	82.13	92.28
Grace, stored f16	2.85	25.30	97.67	191.62	209.60

Table 8: MDOF/s at 206,763 dof, Mooney–Rivlin with Kelvin–Voigt viscosity. Grace scales 69× on 72 cores for the exact apply and 73× for the stored one. The Apple figures peak at eight threads, not ten.

On Grace `fp16` is not worth carrying: at one thread it is slower than both `fp32` and `fp64`, and at 72 threads it is within two per cent of `fp32` while costing four decimal digits. `fp32` is the default on both machines.

9 Status

The split is implemented and reachable from SFEM. `Op` carries `inexact_supported()`, `inexact_update()` and `inexact_apply()`, with defaults that refuse, so no existing operator or caller changes behaviour. `inexact_update` is explicit by design: the stored tangent stays valid until the next call, which is the reuse the split exists for, and `inexact_apply` takes no state at all, so it cannot silently use a stale one.

Three runtime-typed C ABI entry points per material — assemble, apply from a `metric_tensor_t` store, apply from a compressed one — dispatch on `smesh::ElemType` with the scalar type as a `smesh::PrimitiveType`. `linear_elasticity` is generated in-tree with the split enabled, and its operator reproduces its own exact apply to the store's precision on TET4, TET10 and HEX8.

Open:

1. The 2D kernels are generated but unreachable: smesh's adjugate fill has no TRI3 or QUAD4 case, so the affine geometry cache the split reads cannot be built. The operator reports `inexact_supported() == false` there rather than failing at run time, and will start working by itself if smesh gains 2D support.
2. HEX8's contraction is still behind the hand-written kernel by about 1.6×, entirely in the contraction rather than the assembly.
3. The 2D store size is emitted and has never executed.

4. A permanent test belongs in `frontend/tests/`, beside `sfem_PartialAssemblyTest.cpp`, rather than in a spike.